
Plan de pruebas — Evento Cultural

Plan de pruebas y evidencias (artefacto 7 del marco ágil, anexo metodológico)

Roberto Rizzo · José Miguel Moncada (QA) · David Herrera

25 de septiembre de 2026

Proyecto Capstone APT122 · Duoc UC · Sede Maipú

1. Propósito y alcance

Este plan organiza la verificación del proyecto según la exigencia del anexo metodológico (pruebas unitarias, de integración, de rendimiento y de seguridad) y la relaciona con los requisitos no funcionales priorizados en Fase 1 (RNF-01 a RNF-12) y con la verificación ya ejecutada. Declara el estado real: lo ejecutado con su resultado, y lo pendiente sin asumirlo como hecho.

2. Estado actual de las pruebas

Suite	Alcance	Comando
Pruebas unitarias del scraper (14 módulos)	Conectores, normalización, filtro cultural, deduplicación, checkpoints y exportación	<code>python -m unittest discover -s tests -v</code>
Pruebas del frontend (Vitest)	Servicios y lógica de componentes	<code>npm test</code>

Cuadro 1: Suites de pruebas existentes y su ejecución.

3. Cobertura por requisito no funcional

RNF	Requisito	Tipo de prueba	Estado
RNF-01	Interfaz clara e intuitiva	Usabilidad	Pendiente: evaluación con usuarios (ver §5)
RNF-02	Diseño responsive / multiplataforma	Funcional manual	Ejecutada: desbordes revisados en 320–1440 px; verificación manual en navegador y emulador de dispositivos
RNF-03	Tiempos de carga aceptables	Rendimiento	Parcial: skeletons y estados de carga implementados; medición formal pendiente
RNF-04	Protección de datos de usuarios y organizadores	Seguridad	Pendiente en frontend (no hay datos personales: estado mock); RLS de Supabase documentada, validación de políticas pendiente
RNF-04	Control de acceso por rol	Seguridad	Pendiente: autenticación real no conectada

RNF	Requisito	Tipo de prueba	Estado
RNF-06	Disponibilidad del servicio	Disponibilidad	Parcial: workflow de GitHub Actions operativo con ejecución manual y cron diario activo desde el 24-09 (verificado en GitHub Actions: corridas #9–#11 del 25 al 27-09); pendiente una ejecución automática completa tras corregir la prueba dependiente del tiempo (§5.1)
RNF-07	Soporte de crecimiento de datos	Rendimiento	Parcial: paginación/búsqueda indexada no medida con volumen real
RNF-08	Consistencia y normalización	Funcional automatizada	Ejecutada: pruebas del pipeline (normalización y deduplicación) en la suite del scraper
RNF-09	Código mantenible y documentado	Estático	<code>npm run lint</code> sin errores; lint de Python por revisión manual
RNF-10	Contenido en español y contexto chileno	Funcional manual	Ejecutada: interfaz en es-CL; catálogo con comunas y regiones de Chile
RNF-11	Despliegue reproducible	Integración	En curso: workflow del scraper desplegado y en testeo del despliegue (evidencia de GitHub Actions); alojamiento web en Vercel y conversión a app con Capacitor en preparación
RNF-11/12	Accesibilidad web	Accesibilidad	Parcial: contraste AA/AAA calculado, teclado y foco verificados en navegador; prueba con lectores de pantalla pendiente

4. Pruebas pendientes según el anexo

4.1 Planificación de las pruebas pendientes

La planificación se organiza en tres bloques en cascada, porque cada uno habilita al siguiente; el orden respeta la situación real del proyecto (datos de ejemplo en el frontend, integración con Supabase pendiente):

1. **Bloque 1 — Integración de datos (habilita a los demás):** sincronización scraper→Supabase con validación del catálogo resultante (RNF-06/08); conexión real del frontend a Supabase

Tipo (anexo)	Plan para cubrirla
Integración	End-to-end scraper→Supabase→frontend: ejecutar el pipeline con sincronización a Supabase y validar el catálogo resultante; y conexión real del frontend a Supabase (hoy mock), verificando el contrato de <code>recomendacion.model.ts</code>
Rendimiento	Medir tiempos de carga del frontend con el catálogo sincronizado y la consulta a Supabase en volumen; repetir <code>cargarEventos</code> con datos reales
Seguridad	Validar las políticas RLS de Supabase con roles reales (anónimo, autenticado) y revisar el manejo de secretos del workflow
Usabilidad (RNF-01)	Evaluación con usuarios objetivo; puede integrarse con la validación de historias de usuario HU-01–15

Cuadro 3: Pruebas pendientes y cómo se cubrirán.

sustituyendo `data/mock-eventos.ts`, verificando el contrato de `recomendacion.model.ts`. *Criterio de salida:* el catálogo consultado por el frontend proviene de Supabase y las pruebas de la suite del scraper pasan contra el entorno real.

- Bloque 2 — Medición sobre datos reales:** tiempos de carga del frontend (RNF-03), consulta a Supabase con volumen real (RNF-07) y validación de políticas RLS con roles reales (RNF-04/05). *Criterio de salida:* métricas registradas en la §5 con fecha y volumen de eventos.
- Bloque 3 — Validación con usuarios y accesibilidad:** evaluación de usabilidad (RNF-01) integrada con la revisión de HU-01–15, y prueba con lectores de pantalla (RNF-12). *Criterio de salida:* acta de resultados por historia y registro de hallazgos de accesibilidad.

Los bloques 1 y 2 dependen del despliegue/estado del catálogo real; el bloque 3 puede planificarse en paralelo al 2. Cada ejecución se agrega a la §5 sin reescribir el histórico, y el responsable de cada bloque queda **por asignar por el equipo** (roles de referencia: BD/QA para bloques 1 y 2, Frontend/UX para el 3).

5. Registro de resultados

Los resultados de las pruebas ya ejecutadas están documentados en la documentación técnica (Documento_Tecnico_Capstone, sección de pruebas) y en el control de cambios del frontend. Este plan se actualizará con cada ronda de pruebas; los resultados nuevos se agregan sin reescribir el histórico.

5.1 Incidencia de integración continua (27-09-2026)

Fallo registrado en GitHub Actions durante la ejecución automática de la suite del scraper (corrida #11, `scrape-events.yml`, disparo programado del cron diario, 27-09-2026 09:12 UTC, commit 63e310c; verificado contra la API pública de GitHub Actions). Se consigna aquí como registro detallado del incidente (no existe un artefacto separado de incidencias en el repositorio).

Pendiente de completar por el equipo: resultados medibles (tiempos, cobertura, volumen) a medida que se ejecuten las pruebas pendientes de la §4.

5.2 Limitación operativa: capa gratuita de NVIDIA Build

Las pruebas que involucran el enriquecimiento con IA están sujetas a la cuota de la capa gratuita de **NVIDIA Build** (modelos en la nube, 40 peticiones por minuto global): no se puede

Campo	Detalle
Fecha	27-09-2026
Componente	Scraper de eventos culturales
Entorno	GitHub Actions (integración continua), corrida #11 de <code>scrape-events.yml</code>
Tipo	Fallo de prueba automatizada dependiente del tiempo
Severidad	Baja
Impacto	Ninguno en producción; bloqueo del workflow antes de ejecutar la extracción (pasos de extracción y sincronización saltados)
Prueba afectada	<code>test_ceina_cards_are_mapped_to_canonical_event_fields</code>
Resultado	138 pruebas ejecutadas: 137 correctas, 1 fallida (<code>AssertionError: 0 != 1</code>)
Causa raíz	La prueba simulaba un evento de CEINA entre el 24 y el 26-09-2026 y congelaba la fecha del parser en el 21-09 (<code>chile_now</code>); la comprobación de vigencia <code>is_upcoming()</code> de <code>sanitization.py</code> no estaba bajo ese reloj simulado y usó la fecha real de ejecución (27-09), descartando el evento como vencido. Verificado en el código fuente del repositorio: <code>tests/test_event_sources.py</code> y <code>eventos/sanitization.py</code>
Aclaración	No se evidenció caída de CEINA ni fallo de Supabase, NVIDIA o del proceso real de extracción; los mensajes <code>ERROR:root</code> del registro eran excepciones simuladas por pruebas que finalizaron correctamente
Acción correctiva	Congelar en la prueba tanto la fecha utilizada por el parser como la utilizada por <code>is_upcoming()</code> (reloj controlado); recomendación extendida a toda prueba que evalúe vigencia
Estado	Pendiente de corrección: no se marca resuelto sin evidencia de una ejecución posterior exitosa (la corrida del 26-09, #10, sí pasó; la falla específica surgió al vencer las fechas simuladas)
Criterios de cierre	(1) la prueba específica termina correctamente; (2) las 138 pruebas terminan correctamente; (3) GitHub Actions finaliza con código de salida 0
Aprendizaje	Las pruebas que evalúan vigencia de eventos deben utilizar un reloj controlado y no depender de la fecha real del sistema

Cuadro 4: Incidencia de CI: prueba dependiente del tiempo en la suite del scraper.

planificar una batería intensiva de pruebas sobre el pipeline completo sin agotar la cuota ni saturar la latencia. Por ello, las corridas de prueba del pipeline deben planificarse por muestras (subconjuntos de fuentes o de fichas) y aprovechar el mecanismo de checkpoints, que evita reprocesar lo ya completado. Si la ejecución se interrumpe por la cuota, se reanuda sin repetir trabajo.

Pendiente de completar por el equipo: calendario de corridas de prueba (fechas y tamaño de muestra) una vez que el equipo defina la ventana de pruebas del bloque 1.